EWB CLOUD LABS | STUDENT PROJECT REPORT

AWS VPC Peering

Connect two VPCs. Prove private connectivity.

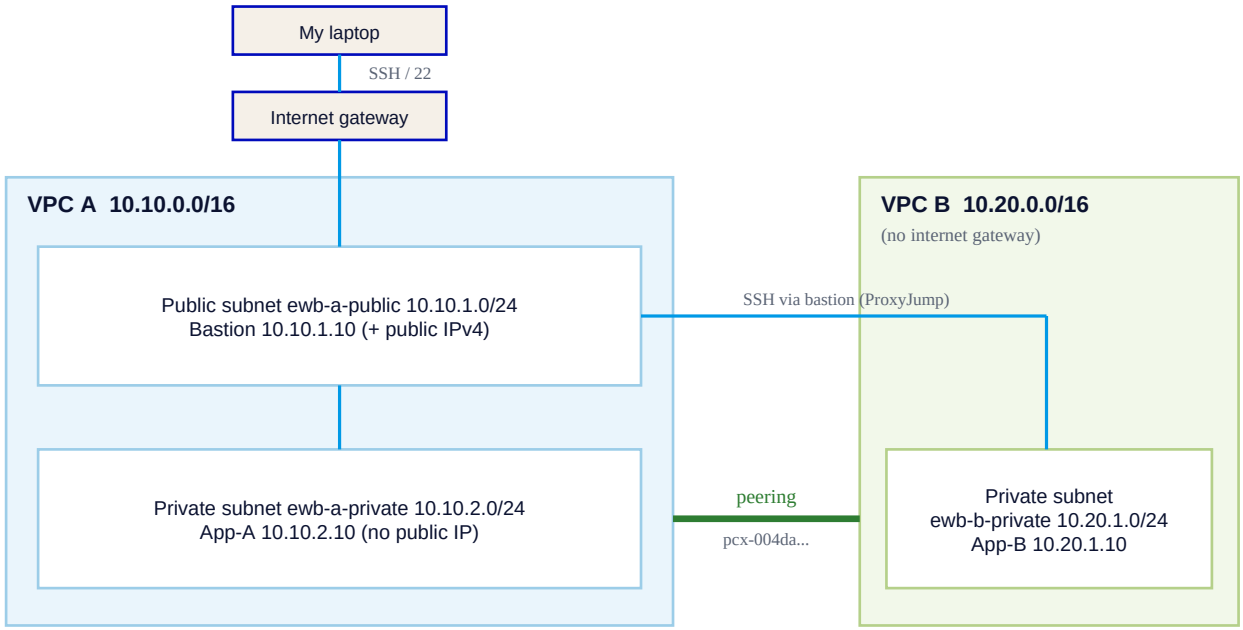
Name	Akshara Ramesh babu
Batch	1
Project	Project 01: VPC Peering (AWS Architect Associate)
Region	Asia Pacific (Sydney), ap-southeast-2
Date	25 October 2026

1. Project summary

This project connects two private VPCs with a VPC peering connection so that servers in each VPC can exchange traffic over private IPv4 addresses. One public bastion host in VPC A is used for administration over SSH. Both application servers have no public IP address. The resources were created with an AWS CLI script run in AWS CloudShell, using the names, CIDR ranges and security rules from the workbook, and then checked with tests run from my laptop and from inside the instances, and with the AWS console screenshots below.

2. Architecture and resource inventory

VPC A (10.10.0.0/16) holds a public subnet with the bastion and a private subnet with App-A. VPC B (10.20.0.0/16) holds a private subnet with App-B and has no internet gateway. The peering connection links the two VPCs. Administration follows the path laptop > bastion > App-A or App-B using SSH ProxyJump.



Architecture diagram (drawn for this report, conceptual).

Resource	Name tag	ID	CIDR / address
VPC A	ewb-vpc-a	vpc-02b2340eddb9f6e95	10.10.0.0/16
Public subnet	ewb-a-public	subnet-0f731d17b03030432	10.10.1.0/24
Private subnet A	ewb-a-private	subnet-0e003b7886964f6c6	10.10.2.0/24
VPC B	ewb-vpc-b	vpc-0610d227ce6642763	10.20.0.0/16
Private subnet B	ewb-b-private	subnet-028a87a99a1b7761a	10.20.1.0/24
Internet gateway	ewb-igw-a	igw-02bd74a5c1b5679d2	Attached to VPC A
Route table, public	ewb-rt-a-public	rtb-0c6257fc2d40f90e0	Associated to ewb-a-public
Route table, private A	ewb-rt-a-private	rtb-0b3f0b170b33177b6	Associated to ewb-a-private
Route table, private B	ewb-rt-b-private	rtb-0f4e1349c68735a5d	Associated to ewb-b-private
Peering connection	ewb-a-b-peer	pcx-004da20d345dee653	VPC A to VPC B
Bastion	ewb-bastion	i-0962d3cc557347d29	10.10.1.10 + public IPv4 3.27.195.30
App-A	ewb-app-a	i-0efa92777ea033a42	10.10.2.10, no public IP
App-B	ewb-app-b	i-0b596295626d8aca6	10.20.1.10, no public IP
Security groups	ewb-sg-bastion ewb-sg-app-a ewb-sg-app-b	sg-050cdd073381ba343 sg-0c21f0a68436d0006 sg-0767dcff632093910	VPC A, VPC A, VPC B

The two VPC CIDRs do not overlap, which peering requires. Both VPCs were checked in the console:

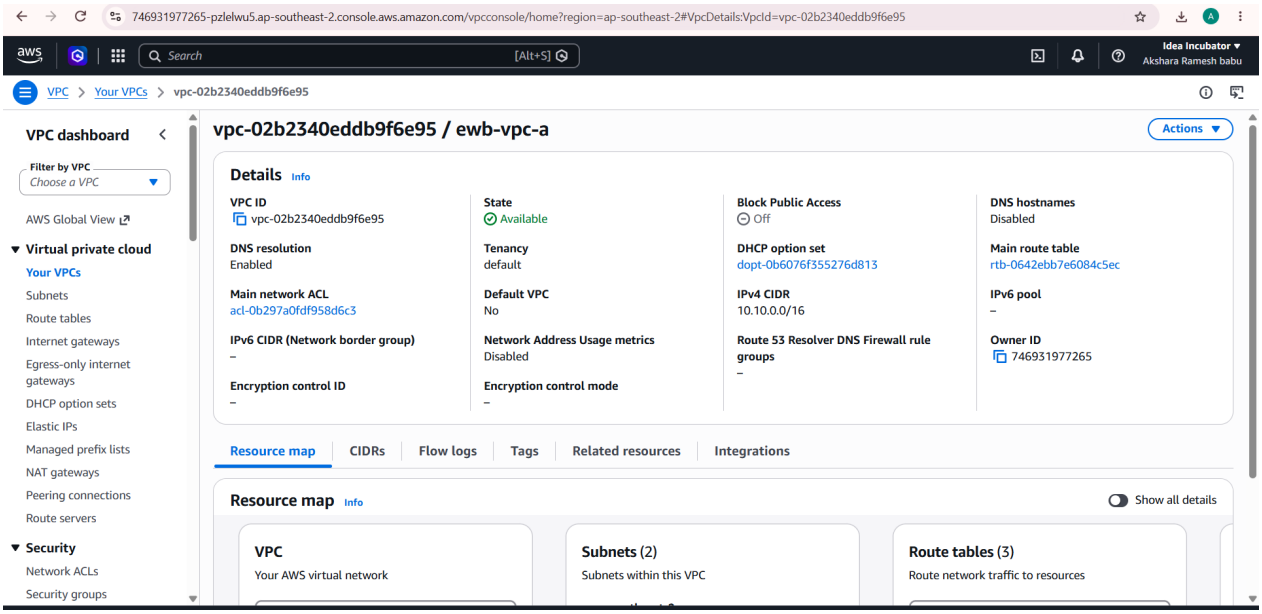


Figure 1. VPC A: ewb-vpc-a (vpc-02b2340eddb9f6e95), IPv4 CIDR 10.10.0.0/16, state Available, with 2 subnets and 3 route tables.

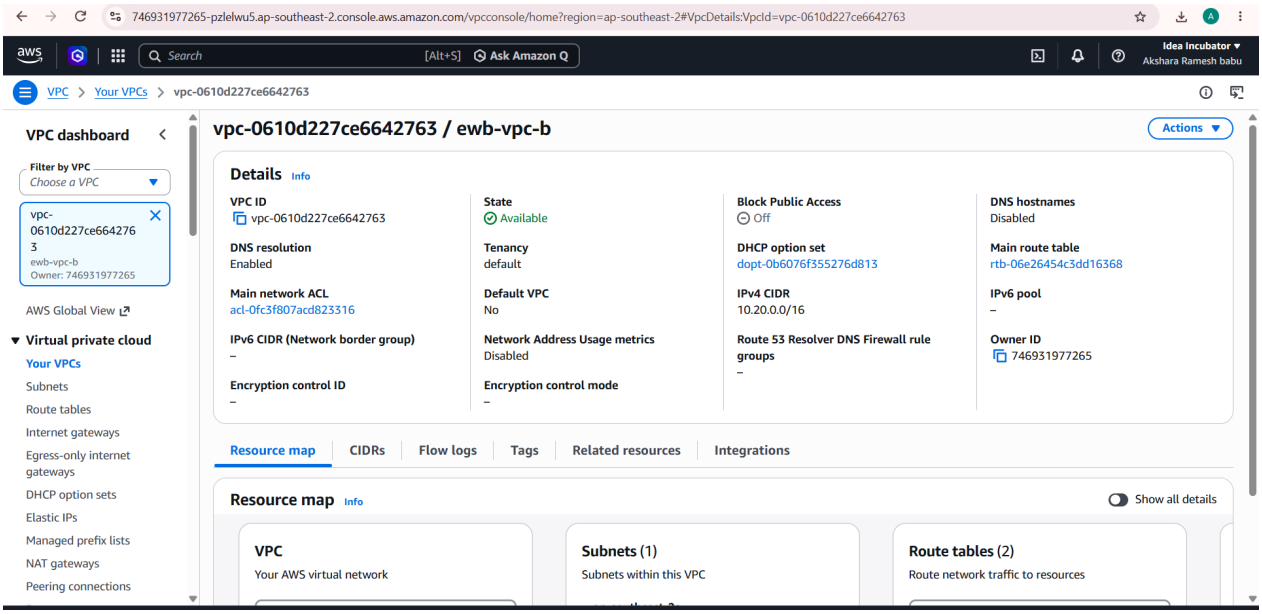


Figure 2. VPC B: ewb-vpc-b (vpc-0610d227ce6642763), IPv4 CIDR 10.20.0.0/16, state Available, with 1 subnet and 2 route tables.

```
ASSOC_APRIV=rtbassoc-081ac4a19154a732a
VPC_B=vpc-0610d227ce6642763
SN_BPRIV=subnet-028a87a99a1b7761a
RT_BPRIV=rtb-0f4e1349c68735a5d
ASSOC_BPRIV=rtbassoc-0a5ba932b553b6866
SG_BASTION=sg-050cdd073381ba343
SG_APPA=sg-0c21f0a68436d0006
SG_APPB=sg-0767dcff632093910
I_BASTION=i-0962d3cc557347d29
I_APPA=i-0efa92777ea033a42
I_APPB=i-0b596295626d8aca6

== Bastion public IPv4: 3.27.195.30 ==
App-A / App-B public IPs (should be None):
i-0efa92777ea033a42     None
i-0b596295626d8aca6     None
```

Figure 3. CloudShell output after the build: resource IDs, the bastion public IPv4 (3.27.195.30), and public IP = None for App-A and App-B.

3. Subnets, internet gateway and route tables

The public subnet is associated with a route table that sends 0.0.0.0/0 to the internet gateway. The private subnets have no internet route. After peering, each table also received a route to the other VPC.

Route table	Destination	Target	Purpose
ewb-rt-a-public	10.10.0.0/16	local	Bastion to App-A
ewb-rt-a-public	0.0.0.0/0	ewb-igw-a (igw-02bd74a5c1b5679d2)	Bastion internet access
ewb-rt-a-public	10.20.0.0/16	pcx-004da20d345dee653	Bastion SSH to App-B
ewb-rt-a-private	10.10.0.0/16	local	VPC A traffic
ewb-rt-a-private	10.20.0.0/16	pcx-004da20d345dee653	App-A to VPC B
ewb-rt-b-private	10.20.0.0/16	local	VPC B traffic
ewb-rt-b-private	10.10.0.0/16	pcx-004da20d345dee653	Return path to VPC A

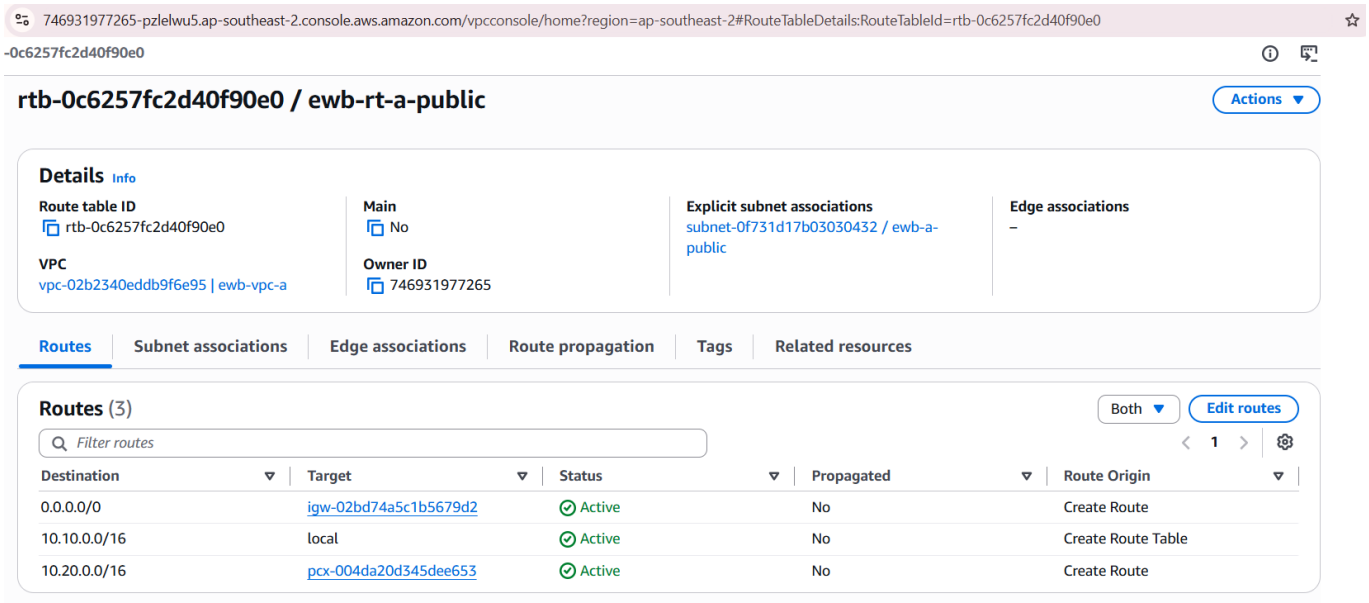


Figure 4. ewb-rt-a-public (rtb-0c6257fc2d40f90e0) in VPC A: routes 0.0.0.0/0 to igw-02bd74a5c1b5679d2, 10.10.0.0/16 local, and 10.20.0.0/16 to pcx-004da20d345dee653, all Active. Explicit subnet association: ewb-a-public (subnet-0f731d17b03030432).

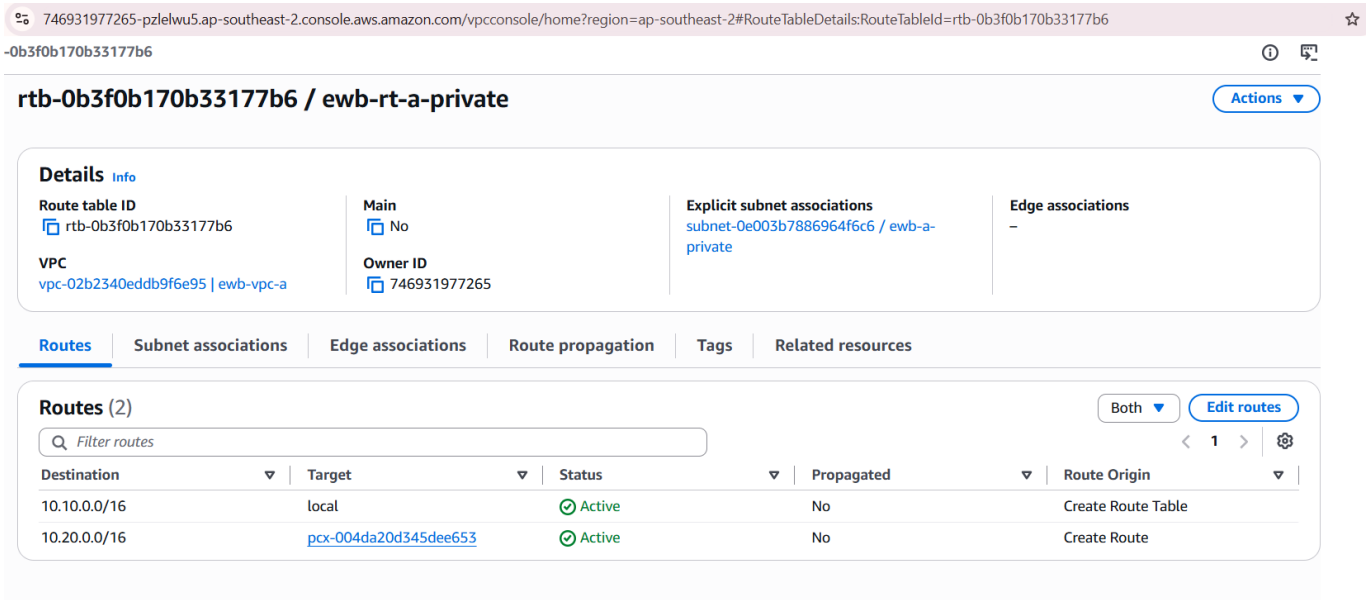


Figure 5. ewb-rt-a-private (rtb-0b3f0b170b33177b6) in VPC A: 10.10.0.0/16 local and 10.20.0.0/16 to pcx-004da20d345dee653 (forward route), both Active. Explicit subnet association: ewb-a-private (subnet-0e003b7886964f6c6). No internet route.

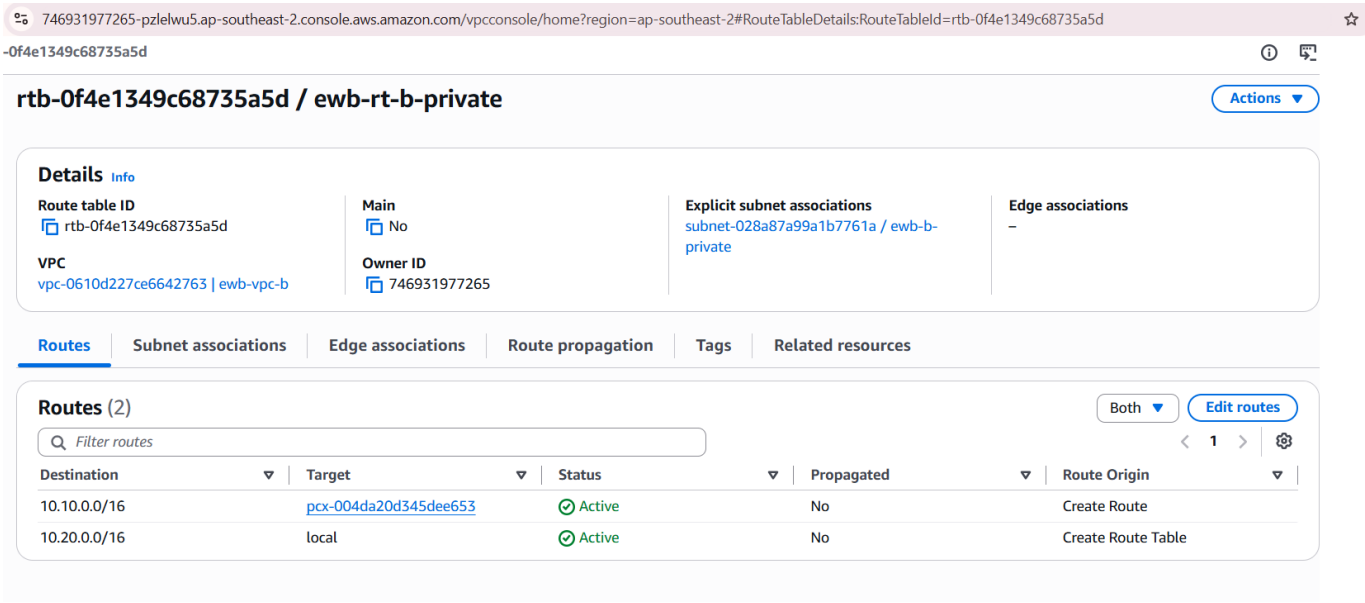


Figure 6. ewb-rt-b-private (rtb-0f4e1349c68735a5d) in VPC B: 10.10.0.0/16 to pcx-004da20d345dee653 (return route) and 10.20.0.0/16 local, both Active. Explicit subnet association: ewb-b-private (subnet-028a87a99a1b7761a).

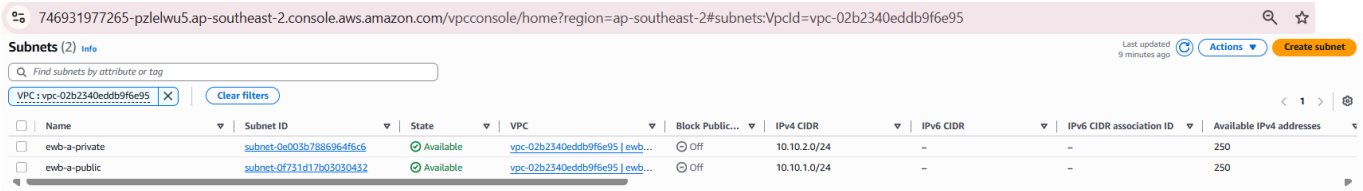


Figure 7. Subnets in VPC A (vpc-02b2340eddb9f6e95): ewb-a-private 10.10.2.0/24 and ewb-a-public 10.10.1.0/24, both Available, auto-assign public IPv4 off.

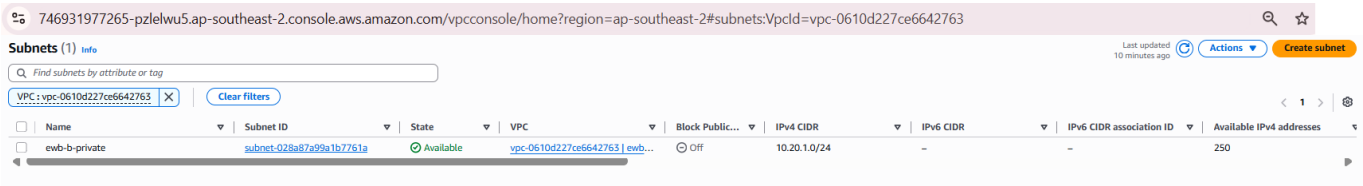


Figure 8. Subnet in VPC B (vpc-0610d227ce6642763): ewb-b-private 10.20.1.0/24, Available.

4. VPC peering connection

The peering request was created from VPC A (requester) to VPC B (accepter) in the same account and Region, then accepted. A peering connection alone does not carry traffic: the routes in section 3 and the security group rules in section 5 decide whether traffic can use it.



Figure 9. Output of the peering step in CloudShell: the connection status is active.

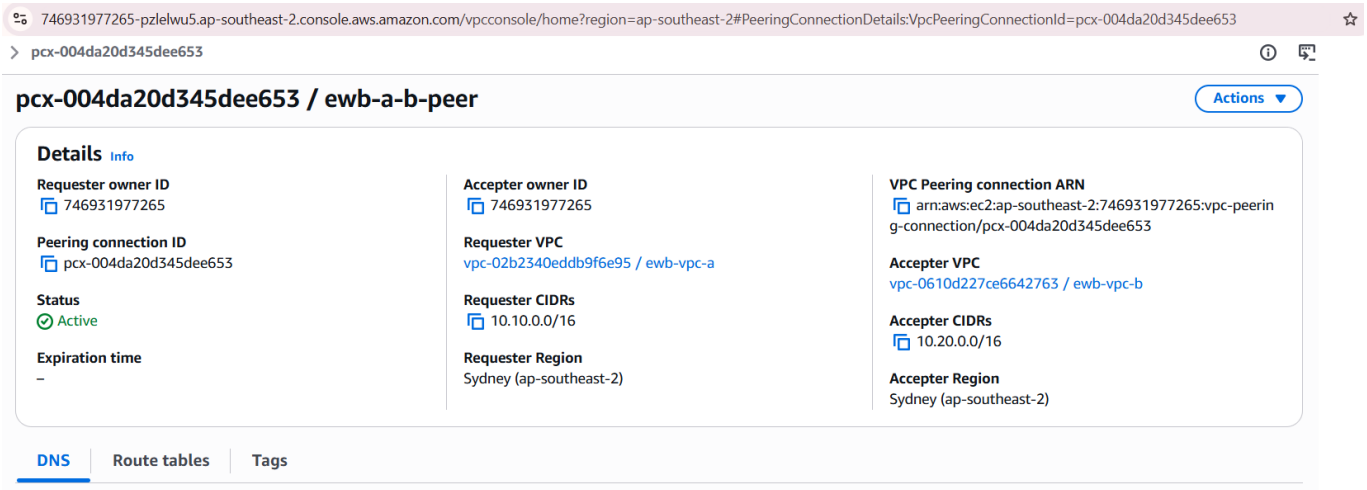


Figure 10. Peering connection ewb-a-b-peer (pcx-004da20d345dee653): status Active. Requester VPC vpc-02b2340eddb9f6e95 (ewb-vpc-a, 10.10.0.0/16), accepter VPC vpc-0610d227ce6642763 (ewb-vpc-b, 10.20.0.0/16), both in Sydney (ap-southeast-2).

5. Security groups

Only the traffic needed for the lab is allowed inbound. Outbound uses the default allow-all rule, and security groups are stateful, so replies to permitted traffic are allowed automatically.

Security group	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	27.60.173.176/32 (my current public IP)
ewb-sg-bastion (VPC A)	SSH / TCP 22	106.192.170.33/32 (address entered when the group was first built)
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	ICMP Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	ICMP Echo request	10.10.2.10/32 (App-A)

My public IP address changed during the project, so I added my current address as a second SSH rule when SSH to the bastion first timed out. The earlier address is harmless and is removed with the group during cleanup.

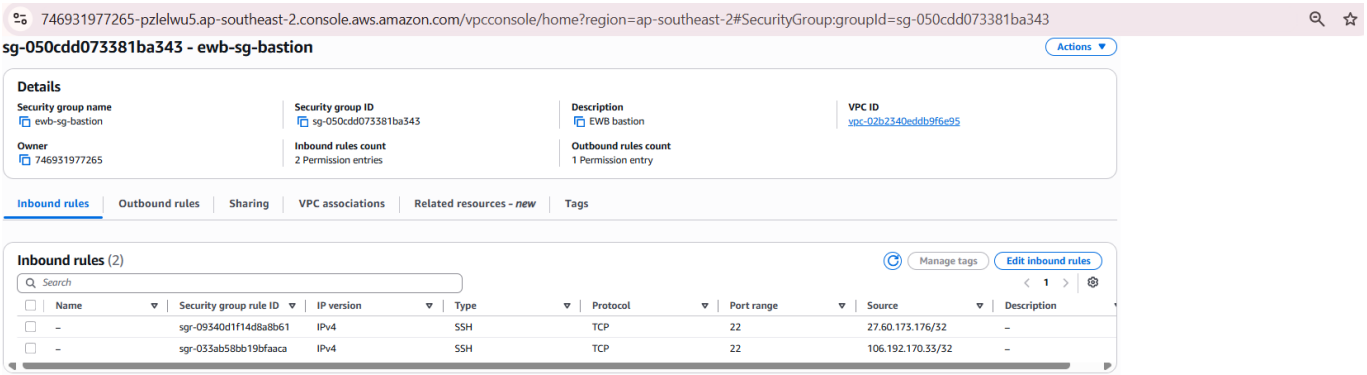


Figure 11. ewb-sg-bastion (sg-050cdd073381ba343), VPC A: inbound SSH, TCP 22, from 27.60.173.176/32 and 106.192.170.33/32.

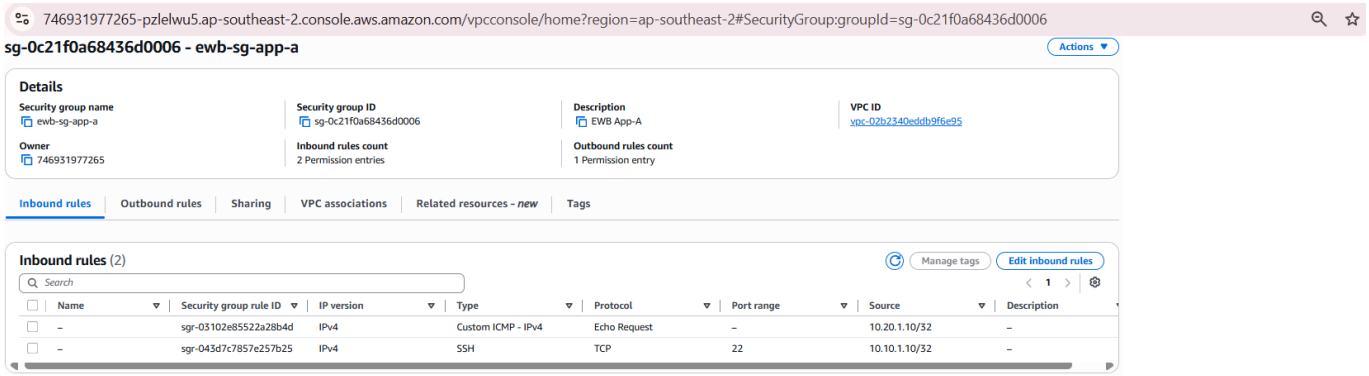


Figure 12. ewb-sg-app-a (sg-0c21f0a68436d0006), VPC A: inbound Custom ICMP Echo Request from 10.20.1.10/32 and SSH, TCP 22, from 10.10.1.10/32.

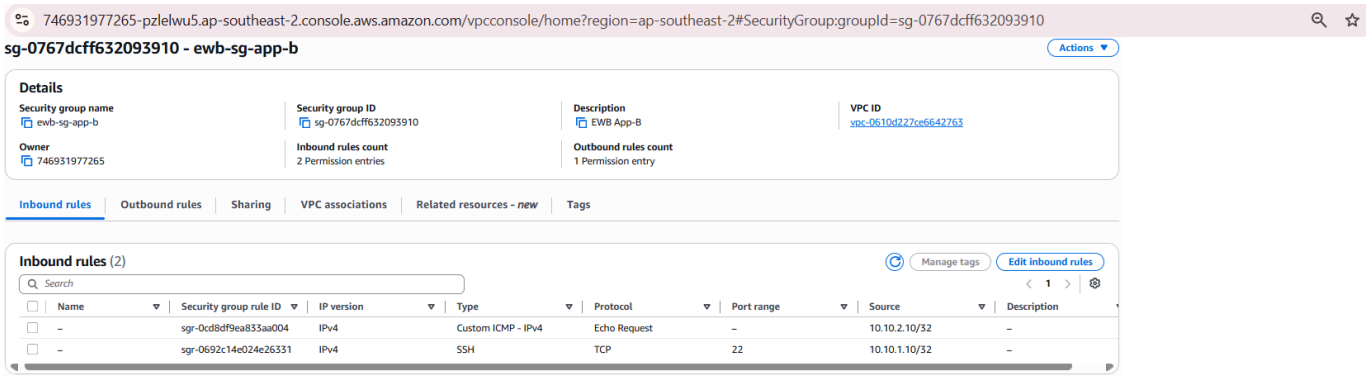


Figure 13. ewb-sg-app-b (sg-0767dcff632093910), VPC B: inbound Custom ICMP Echo Request from 10.10.2.10/32 and SSH, TCP 22, from 10.10.1.10/32.

6. Instances and isolation

The CloudShell output in Figure 3 shows no public IPv4 address for App-A and App-B. The console views below confirm it: the bastion is the only instance with a public IPv4 address.

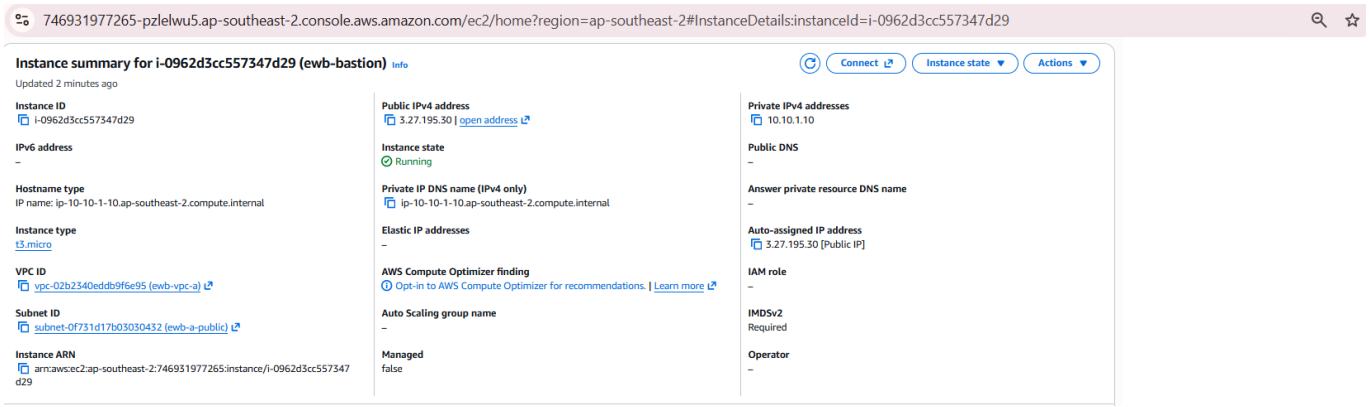


Figure 14. ewb-bastion (i-0962d3cc557347d29): Running, private IPv4 10.10.1.10, public IPv4 3.27.195.30, VPC A, subnet ewb-a-public.

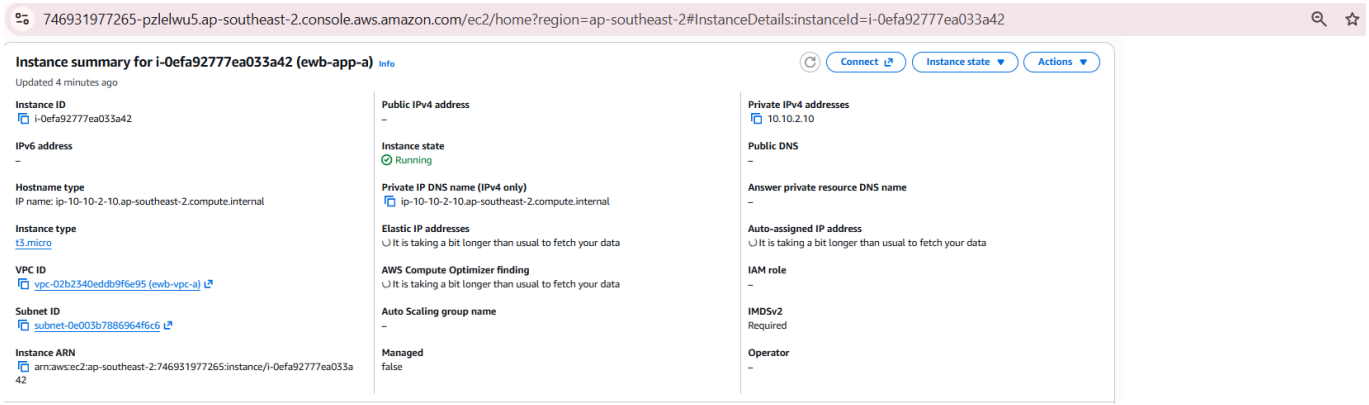


Figure 15. ewb-app-a (i-0efa92777ea033a42): Running, private IPv4 10.10.2.10, Public IPv4 address empty, VPC A (vpc-02b2340eddb9f6e95), subnet subnet-0e003b7886964f6c6.

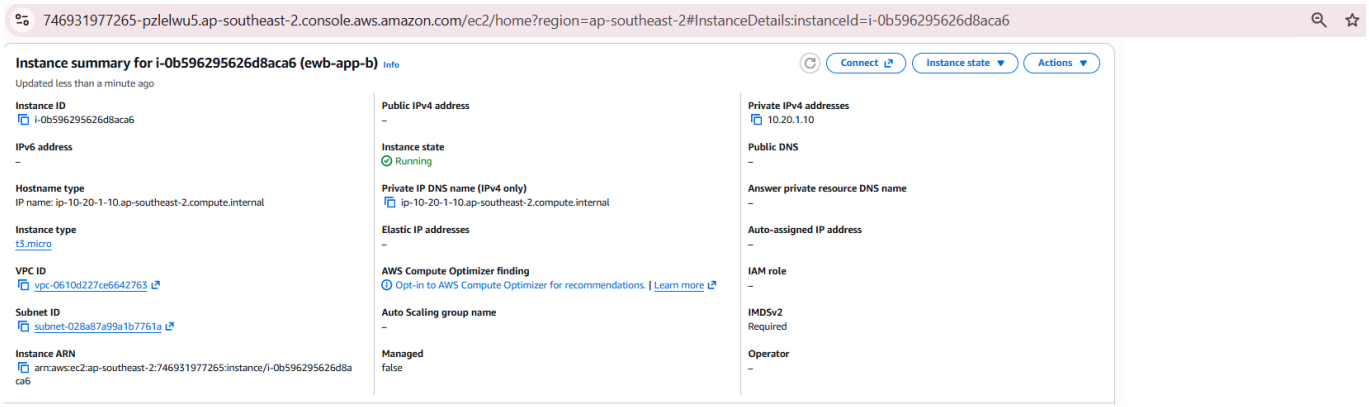


Figure 16. ewb-app-b (i-0b596295626d8aca6): Running, private IPv4 10.20.1.10, Public IPv4 address empty, VPC B (vpc-0610d227ce6642763), subnet subnet-028a87a99a1b7761a.

7. Test evidence

All pings were run inside the instances (the prompt shows the host), and SSH was run from my laptop through the bastion.

Test	Where / command	Expected	Result
01 Baseline, before peering	App-A: ping -c 4 10.20.1.10	No replies	Done: 0 received, 100% packet loss (Figure 17)
02 Forward path	App-A: ping -c 4 10.20.1.10	4 replies, 0% loss	Done: 4 received, 0% packet loss (Figure 18)
03 Reverse path	App-B: ping -c 4 10.10.2.10	4 replies, 0% loss	Done: 4 received, 0% packet loss (Figure 19)
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Done: prompt ip-10-20-1-10 reached (Figure 19)
05 Isolation	App-A and App-B public IPv4	None	Done: field empty in the console (Figures 15 and 16) and None in CloudShell (Figure 3)

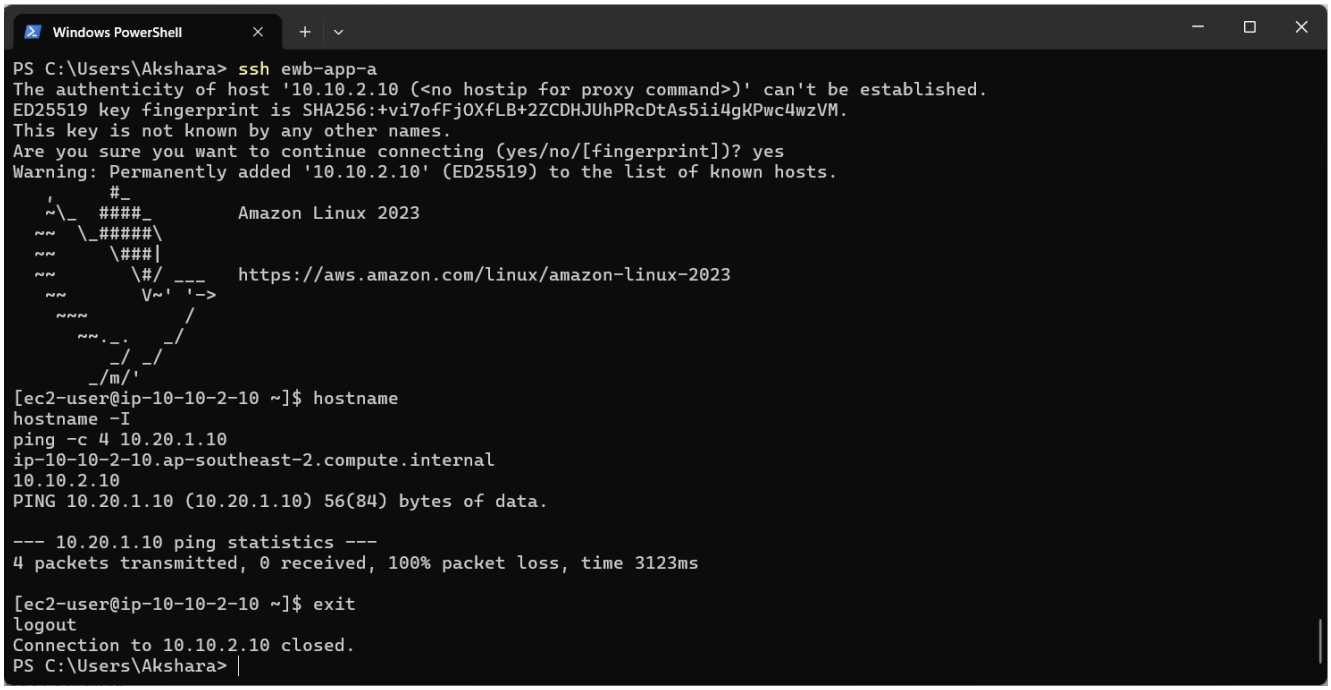


Figure 17. Test 01, baseline before peering. App-A (ip-10-10-2-10): ping -c 4 10.20.1.10 gives 4 transmitted, 0 received, 100% packet loss. VPC A and VPC B were isolated.

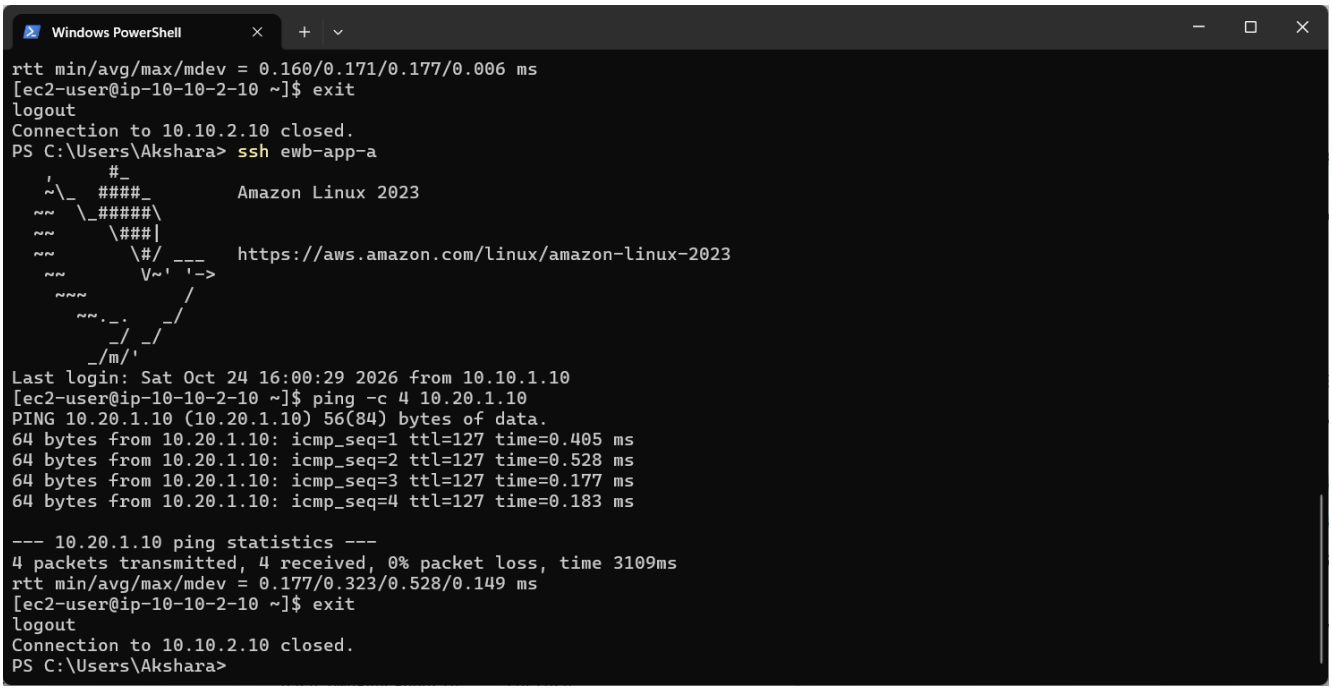


Figure 18. Test 02, forward path after peering. App-A (ip-10-10-2-10): ping -c 4 10.20.1.10 gives 4 transmitted, 4 received, 0% packet loss.

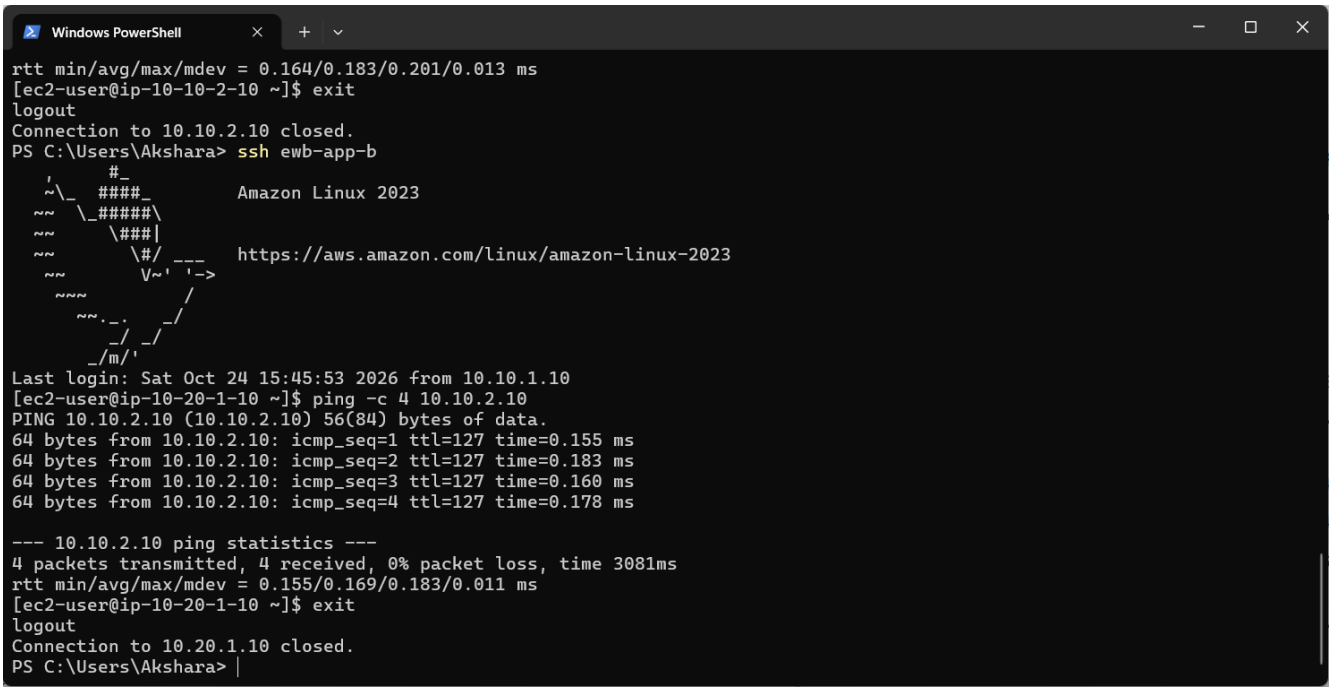


Figure 19. Tests 03 and 04. ssh ewb-app-b reaches App-B through the bastion (prompt ip-10-20-1-10); ping -c 4 10.10.2.10 gives 4 received, 0% packet loss.

8. Controlled failure and recovery

To prove the destination security group is what permits the ping, I removed only the ICMP Echo request rule (source 10.10.2.10/32) from ewb-sg-app-b and left the SSH rule in place. The route and the peering connection stayed active the whole time, so any failure comes from the security group and not from routing.

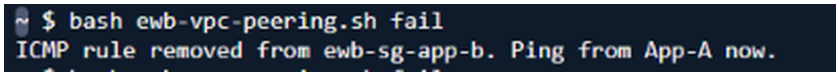


Figure 20. Step 1. CloudShell: bash ewb-vpc-peering.sh fail removes the ICMP Echo request rule from ewb-sg-app-b.

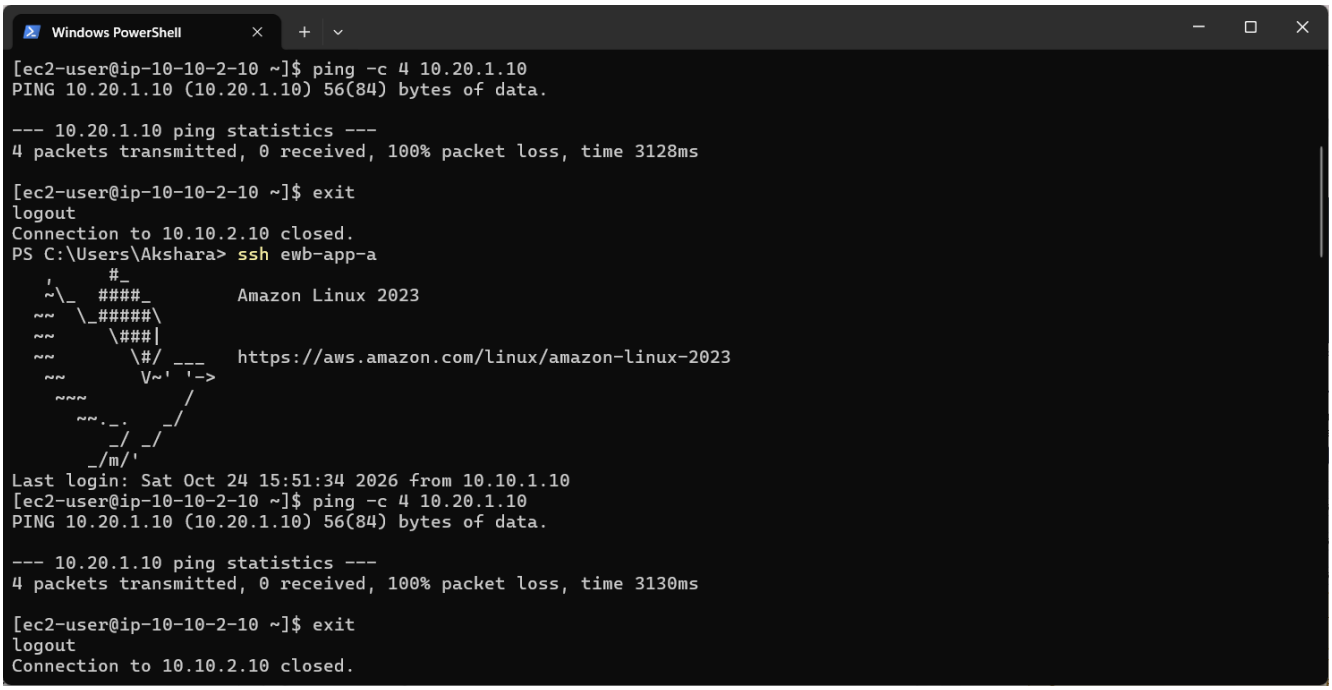


Figure 21. Step 2. App-A (ip-10-10-2-10): with the rule removed, ping -c 4 10.20.1.10 gives 4 transmitted, 0 received, 100% packet loss.

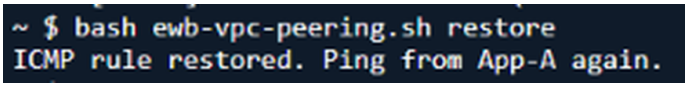


Figure 22. Step 3. CloudShell: bash ewb-vpc-peering.sh restore puts the same ICMP rule back.

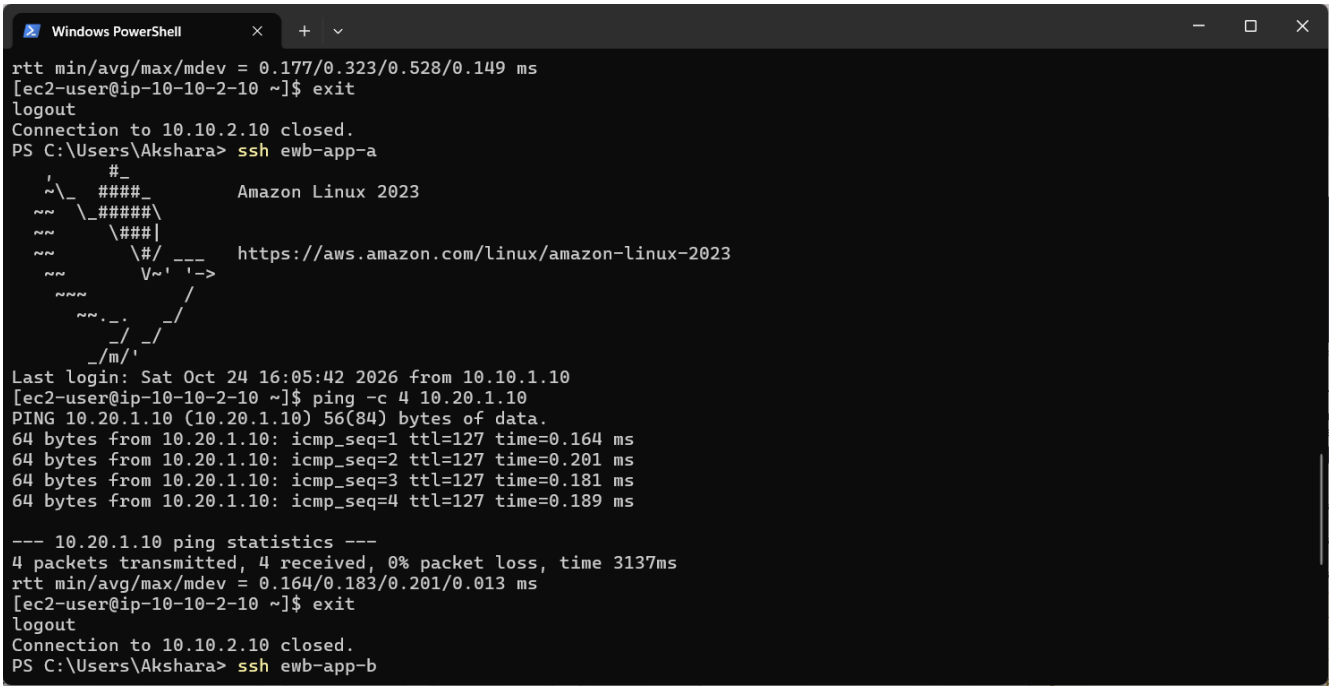


Figure 23. Step 4. App-A (ip-10-10-2-10): after the rule is restored, ping -c 4 10.20.1.10 gives 4 transmitted, 4 received, 0% packet loss.

9. Forward path and return path

Forward path. A ping from App-A (10.10.2.10) to App-B (10.20.1.10) matches the 10.20.0.0/16 route in ewb-rt-a-private, which points at the peering connection. The packet enters VPC B, where ewb-sg-app-b allows ICMP Echo request from 10.10.2.10/32, so it reaches App-B.

Return path. App-B replies to 10.10.2.10. That matches the 10.10.0.0/16 route in ewb-rt-b-private, which also points at the peering connection, so the reply crosses back to VPC A. Security groups are stateful, so App-A needs no extra inbound rule for the reply.

If either route table lacked its route, the request or the reply would have no path, and the ping would fail even though the peering shows Active.

Public subnet peering route. With ProxyJump, the bastion opens its own SSH connection to App-B, so its subnet needs 10.20.0.0/16 pointing at the peering connection (Figure 4). This more specific route is chosen instead of the internet default route. For the same reason, the App-B SSH rule uses the bastion private IP (10.10.1.10/32) as its source, not my laptop address.

10. Cleanup

After saving all the evidence above, I ran the cleanup command of my script. It terminated the three instances, deleted the peering connection, the route tables, security groups, subnets and internet gateway, deleted both VPCs, and removed the key pair, all of which were created for this project. I then checked the consoles.

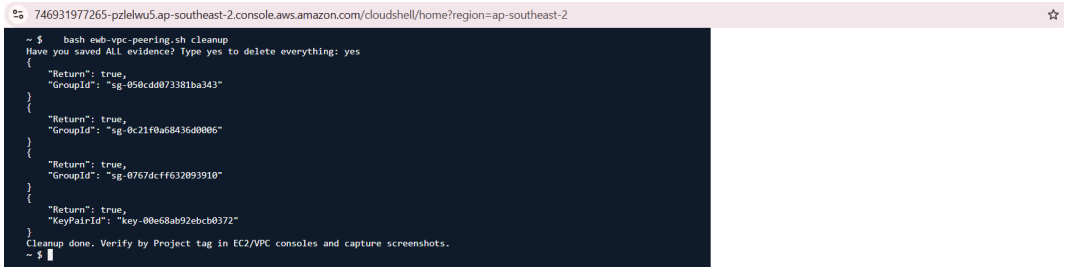


Figure 24. Output of bash ewb-vpc-peering.sh cleanup in CloudShell: the three security groups (sg-050cdd073381ba343, sg-0c21f0a68436d0006, sg-0767dcff632093910) and the key pair were deleted, and the script finished with "Cleanup done."

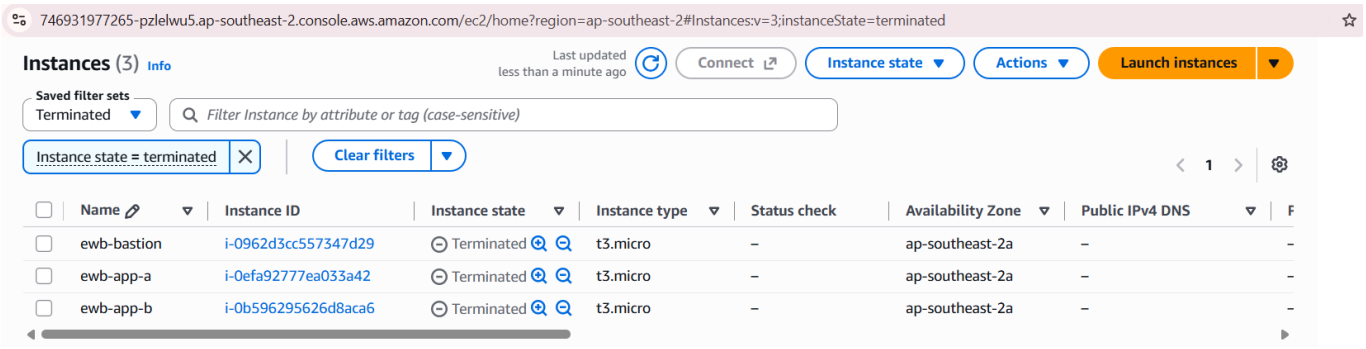


Figure 25. EC2 Instances filtered by Instance state = terminated: ewb-bastion (i-0962d3cc557347d29), ewb-app-a (i-0efa92777ea033a42) and ewb-app-b (i-0b596295626d8aca6) are all Terminated.

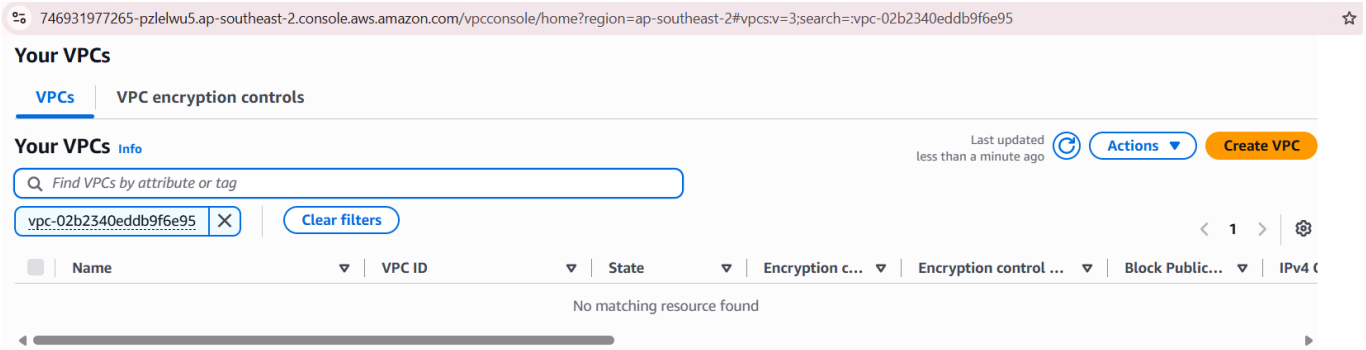


Figure 26. Your VPCs searched for vpc-02b2340eddb9f6e95 (VPC A): No matching resource found, so VPC A is deleted.

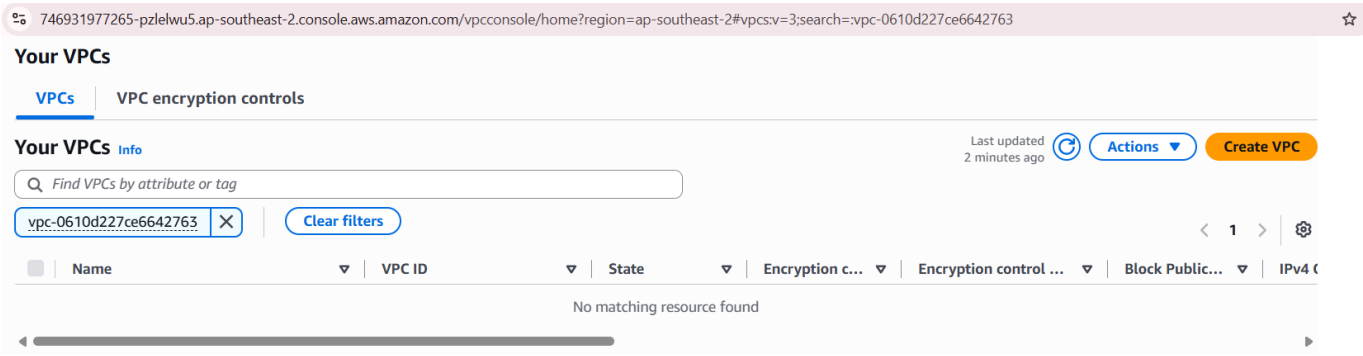


Figure 27. Your VPCs searched for vpc-0610d227ce6642763 (VPC B): No matching resource found, so VPC B is deleted.

11. What I learned

- A peering connection is only the link. Routes on both sides and security group rules decide whether traffic passes.
- The failed baseline has to be captured before peering is created, because it cannot be recreated later without undoing the routes.
- My public IP changed during the project, which made SSH to the bastion time out until I added my current address to its security group. Comparing my current IP with the rule located the problem.
- Linux commands such as ping -c 4 must be run inside the instance. Run on my Windows laptop after leaving the instance, the same command is rejected.
- The default limit of five VPCs per Region stopped one build attempt, so I had to check which resources my commands were actually using before taking screenshots.